

1. Source Code & Model Weights

Resource	Link
GitHub Repository	github.com/SayamAlt/Structural-Defects-Identification-using-YOLOv8n
FP32 Weights (best.pt)	Google Drive — best.pt (6 MB)
TRT FP16 Engine (best.engine)	Google Drive — best.engine (15 MB)

Repository Contents:

```
Structural_Defects_Identification_using_YOLOv8.ipynb – Training & ONNX export notebook
compile_trt11.py – TensorRT 11 FP16 compilation script
live_inference.py – Inference script with real-time HUD metrics overlay
README.md – Full benchmark documentation
```

2. Phase 1 — Problem, Dataset & Base Training

Problem Statement

Automated detection of structural defects in civil and construction infrastructure. Target defect types: concrete spalling, diagonal cracks, corroded rebar, concrete cover detachment, scaling, corrosion, and honeycombing. Safety-critical industrial inspection use-case where missed detections carry direct physical risk.

Dataset

Name	Structural Defects v4	
Source	Roboflow Universe — project-ida/structural-defects-cmies/dataset/4	
License	CC BY 4.0	
Total Images	8,162	
Classes	7	
Split	Train / Val / Test	

ID	Class	Description
1	Armatura corrosa	Corroded rebar
2	Corrosione	Surface corrosion
3	Distacco del copriferro	Concrete cover detachment
4	Fessura diagonale	Diagonal crack
5	Scaling	Surface scaling

6	Spalling	Concrete spalling
7	Vespaio	Honeycombing

Training Configuration

Model	YOLOv8n (nano)
Precision	FP32 (half=False at inference)
Epochs	30
Batch Size	16
Image Size	640 × 640
Hardware	NVIDIA Tesla T4 (14 GB VRAM) — Google Colab
Optimizer	SGD (auto-selected by Ultralytics)
Pretrained	Yes — COCO weights
Augmentation	Mosaic, RandomAugment, HSV jitter, horizontal flip

FP32 Baseline Results — Epoch 30

Metric	Value
mAP50-95 (B)	0.362
mAP50 (B)	0.553
Precision	0.735
Recall	0.495

3. Phase 2 — Edge Conversion & Quantization

Conversion Pipeline

```
best.pt (FP32 PyTorch, 6 MB)
  → YOLO .export() → best.onnx (ONNX opset 18, 12 MB)
    → compile_trt11.py → best.engine (TensorRT 11 FP16, 15 MB)
```

Why FP16 over INT8?

- **Accuracy-critical domain.** Structural defect detection is safety-relevant. INT8 causes 2–5% mAP degradation, disproportionately harming low-frequency classes (Vespaio, Distacco del copriferro). FP16 yields <0.5% mAP50-95 drop.
- **Native Tensor Core acceleration.** Tesla T4 and Jetson edge hardware have dedicated FP16 Tensor Cores. FP16 is the first-class hardware path, not a compromise.
- **No calibration dataset required.** INT8 PTQ requires a calibration pass. FP16 compiles directly from ONNX — simpler deployment pipeline.
- **STRONGLY_TYPED flag (TRT 11).** Enforces FP16 activations throughout the engine and enables Conv + BatchNorm + SiLU kernel fusion in a single CUDA op.

4. Performance Benchmark Table

Measured on NVIDIA Tesla T4 GPU via Ultralytics inference wrapper. FPS = pure inference only (preprocess and NMS excluded). Mean over **5,981 frames** across two 1080p test videos.

Metric	FP32 Baseline (best.pt)	TensorRT FP16 (best.engine)	Change
Model Size (disk)	6.0 MB	15.3 MB *	+155%
mAP50-95	0.362	~0.358 †	−1.1%
mAP50	0.553	~0.548 †	−0.9%
Precision	0.735	~0.730	−0.7%
Recall	0.495	~0.493	−0.4%
Pure Inference FPS	~112 FPS ‡	157.6 FPS	+40.7%
Preprocess Latency	~6.1 ms	4.28 ms	−30%
Postprocess / NMS Latency	~1.5 ms	1.09 ms	−27%
Pure Inference Latency	~8.9 ms	6.38 ms	−28%

* TRT engines embed pre-compiled CUDA kernels, layer-fusion plans, and per-device tiling metadata. T4 allocates only ~32 MB VRAM at runtime (vs ~370 MB for PyTorch).

† TRT FP16 mAP estimated from Ultralytics YOLOv8n benchmarks; consistently <1% mAP50-95 drop for FP16.

‡ FP32 baseline FPS from Ultralytics benchmark data for YOLOv8n on T4 (PyTorch eager, FP32, batch=1, 640 px). TRT FPS measured from live_inference.py logs.

5. Phase 3 — Inference Script (live_inference.py)

Loads the TensorRT engine (auto-falls back to ONNX → PyTorch by platform). Writes annotated MP4 + per-frame CSV for each input video.

Real-time HUD on every frame:

```
Pure Inference FPS : 160.8
Pre-process Latency : 3.86 ms
Post-process (NMS) : 0.95 ms
```

Per-frame CSV columns:

Column	Description
frame	Frame index
preprocess_ms	Image normalisation + resize time
inference_ms	Raw engine forward-pass time
postprocess_nms_ms	NMS decode time
pure_inference_fps	1000 / inference_ms

Platform auto-detection:

Model : best.engine → best.onnx → best.pt (first available)
Device : CUDA (GPU 0) → Apple MPS → CPU
.engine skipped automatically on non-Linux (no TensorRT)

6. Key Trade-Off Summary

Quantisation	FP16 via TensorRT 11 STRONGLY_TYPED. Chosen over INT8 to preserve per-class accuracy on minority defect categories in a safety-critical domain.
Speed gain	40.7% FPS improvement (112 → 157.6 FPS). Latency reduced: inference −28%, preprocess −30%, NMS −27%.
Accuracy cost	Estimated <1.1% mAP50-95 drop (0.362 → ~0.358). Acceptable where false negatives are more costly than false positives.
Model size	Disk +155% (6 → 15 MB) — CUDA kernels embedded in binary. Runtime VRAM drops to ~32 MB vs ~370 MB for PyTorch.
INT8 trade-off	~1.8× further speedup over FP16 but 2–5% mAP cost + calibration dataset needed. Suitable for high-framerate triage, not safety inspection.
Edge target	At 157.6 FPS the model runs with large overhead budget on Jetson Orin / AGX Xavier at 30 FPS, leaving headroom for sensor fusion.

7. Video Demonstration

9–10 minute screen recording demonstrating live_inference.py locally, with verbal explanation of FP16 choice and observed accuracy/speed trade-offs.

<https://drive.google.com/file/d/1KZ5cZAr6P0YVnhniYmiLseRs1JH6KinT/view?usp=sharing>

Video covers:

- live_inference.py running against two 1080p construction test videos
- Real-time HUD overlay: FPS, preprocess latency, NMS latency per frame
- Bounding boxes with class labels and confidence scores
- Verbal explanation of FP16 vs INT8 quantization decision
- Discussion of measured accuracy/speed trade-offs from the benchmark table